📚 GIÁO TRÌNH VEO AUTOMATION — Phần 2 (Chương VI - XI)

CHƯƠNG VI: Content Script — "Tay Robot" Thao Tác DOM

6.1 Content Script là gì?

> Content Script = đoạn code JavaScript được **tự động chèn vào** trang Google Flow. Nó có thể "nhìn thấy" và "sờ được" mọi thứ trên trang — giống như một **con rối robot** đang ngồi trước máy tính và thao tác thay bạn.

Khác biệt quan trọng — Isolated World:

```
┌──────────────────────────────────────────────────────────┐
│ Tab: labs.google.com/fx/tools/flow                       │
│                                                          │
│  ┌────────────────────┐  ┌──────────────────────┐       │
│  │  JS World của trang │  │  Isolated World      │       │
│  │  (Google Flow React) │  │  (Content Script)    │       │
│  │                    │  │                      │       │
│  │  • window.fetch()  │  │  • document.querySelector ✅│
│  │  • React state     │  │  • chrome.runtime ✅  │
│  │  • API responses   │  │  • window.fetch ❌    │
│  │                    │  │    (bản copy riêng!)  │
│  └────────────────────┘  └──────────────────────┘       │
│          ↕ Cùng DOM ↕                                    │
│  ┌──────────────────────────────────────────────────┐   │
│  │         DOM — cả hai đều đọc/ghi được            │   │
│  └──────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

6.2 Kỹ thuật #1: waitForElement — Chờ phần tử xuất hiện

Vấn đề: Google Flow là React SPA — các phần tử **không có sẵn** khi trang mở. Chúng được render **bất đồng bộ** (async). Nếu code chạy `document.querySelector('textarea')` ngay → trả về `null`!

```
function waitForElement(selector, timeout = 15000) {
  return new Promise((resolve, reject) => {
    // Bước 1: Check ngay — có thể element đã tồn tại
    const existing = document.querySelector(selector);
    if (existing) return resolve(existing);

    // Bước 2: Dùng MutationObserver — "người canh gác DOM"
    const observer = new MutationObserver(() => {
      const el = document.querySelector(selector);
      if (el) {
        observer.disconnect(); // Tìm thấy → ngừng canh
        resolve(el);
      }
    });

    // Theo dõi TOÀN BỘ thay đổi trên DOM
    observer.observe(document.body, {
      childList: true,  // Thêm/xóa elements
      subtree: true,    // Tất cả con cháu, không chỉ con trực tiếp
      attributes: true  // Thay đổi class, style, data-*
    });

    // Timeout: nếu 15 giây vẫn không thấy → báo lỗi
    setTimeout(() => {
      observer.disconnect();
      reject(new Error(`Không tìm thấy "${selector}" sau ${timeout}ms`));
    }, timeout);
  });
}
```

> ℹ️ **NOTE**
>
> **MutationObserver** giống như đặt **camera giám sát** trên DOM. Mỗi khi có bất kỳ thay đổi nào (React render xong 1 component), camera thông báo → ta check lại. **Hiệu quả hơn** việc dùng `setInterval` hay `setTimeout` vì phản ứng **ngay lập tức**.

6.3 Kỹ thuật #2: React Input Hacking — Bí kíp quan trọng nhất

Vấn đề cực kỳ quan trọng:

```
// ❌ KHÔNG HOẠT ĐỘNG! React sẽ KHÔNG nhận ra thay đổi
document.querySelector('textarea').value = "my prompt";
// → Textarea hiển thị text, nhưng React state vẫn = "" (rỗng)!
// → Khi bấm Generate, React gửi prompt rỗng lên server!
```

Tại sao? React dùng **Controlled Components** — nó "chiếm quyền" setter `value` của DOM element. Khi bạn set `textarea.value = "..."`, React (ngồi ở giữa) chặn lại: "Tôi không cho phép, vì state của

tôi vẫn là rỗng!".

Giải pháp — Native Setter Trick:

```
function setReactInputValue(element, value) {
  // Bước 1: Tìm setter GỐC của HTML (bỏ qua React override)
  const proto = element instanceof HTMLTextAreaElement
    ? HTMLTextAreaElement.prototype
    : HTMLInputElement.prototype;

  const nativeSetter = Object.getOwnPropertyDescriptor(proto, 'value').set;
  //                    ↑ Lấy setter từ prototype GỐC của HTML
  //                      React override setter trên instance, không phải prototype

  // Bước 2: Gọi setter gốc — "lách" qua React
  nativeSetter.call(element, value);

  // Bước 3: Gửi events để React cập nhật state
  element.dispatchEvent(new Event('input', { bubbles: true }));
  element.dispatchEvent(new Event('change', { bubbles: true }));

  // Bước 4: Bổ sung keyboard events (một số React version cần)
  element.dispatchEvent(new KeyboardEvent('keydown', { bubbles: true }));
  element.dispatchEvent(new KeyboardEvent('keyup', { bubbles: true }));
}
```

Ví dụ dễ hiểu:

```
React giống BẢO VỆ đứng trước cổng nhà (textarea):
  → Bạn đưa thư (set value) → Bảo vệ nói "Không nhận, tôi không biết thư này"
  → Native Setter = bạn LEO TƯỜNG vào nhà (bypass bảo vệ)
  → dispatchEvent = bạn BẤM CHUÔNG ("Có thư mới!") → React quay lại check → "Ơ, đúng l
```

6.4 Kỹ thuật #3: Selector Strategy — Chống thay đổi DOM

Google Flow thay đổi class names mỗi lần deploy (CSS Modules hash). Extension dùng **fallback chain**:

```
const SELECTORS = {
  generateButton: [
    '[aria-label*="Generate"]',              // Ưu tiên 1: Semantic HTML ✅
    '[data-testid="generate-btn"]',          // Ưu tiên 2: Test attribute ✅
    '[data-action="generate"]',              // Ưu tiên 3: Data attribute ✅
    // Ưu tiên 4: Tìm bằng text content (function)
    () => [...document.querySelectorAll('button')]
      .find(b => b.textContent.match(/Generate|Create|Submit/i)),
    '.css-1a2b3c'                            // CUỐI CÙNG: Class hash ❌ (dễ hỏng)
  ]
};

// Thử từng selector từ trên xuống, dừng khi tìm thấy
async function findElement(selectorList) {
```

```
  for (const sel of selectorList) {
    try {
      if (typeof sel === 'function') {
        const el = sel();
        if (el) return el;
      } else {
        const el = await waitForElement(sel, 3000);
        if (el) return el;
      }
    } catch { /* thử selector tiếp theo */ }
  }
  throw new Error('Không tìm thấy element nào!');
}
```

6.5 Luồng automation chính — 9 bước

```
EXECUTE_AUTOMATION(job):
  │
  ▼ Bước 1: Click "New Project" → reactClick(newBtn) → sleep(2000)
  │
  ▼ Bước 2: Chọn mode (Text to Video / Image to Video / ...)
  │
  ▼ Bước 3: Upload ảnh tham chiếu (nếu mode cần ảnh)
  │
  ▼ Bước 4: Điền prompt → setReactInputValue(textarea, job.prompt)
  │
  ▼ Bước 5: Chọn model (Veo 3.1 Fast / Quality / Veo 2...)
  │
  ▼ Bước 6: Chọn tỉ lệ khung hình (16:9 / 9:16 / 1:1)
  │
  ▼ Bước 7: Set duration (5s / 8s)
  │
  ▼ Bước 8: Click "GENERATE" → Google Flow bắt đầu render
  │
  ▼ Bước 9: POLLING — Check mỗi 3 giây cho đến khi:
      ├── ✅ Video xuất hiện → trả về videoUrl
      ├── ❌ Lỗi xuất hiện → throw Error
      └── ⏰ >10 phút → Timeout
```

CHƯƠNG VII: MAIN World Injection — "Gián Điệp" Bắt API

7.1 Tại sao cần inject vào MAIN World?

> ⚠️ **WARNING**

> Content Script chạy trong **Isolated World** — nó CÓ THỂ đọc DOM, nhưng **KHÔNG THỂ** đọc biến JavaScript của Google Flow. Nó cũng **KHÔNG THỂ** intercept (chặn) `window.fetch()` gốc của trang.

Để lấy URL video từ Google API response → cần inject script vào **MAIN World** (cùng không gian JS với Google Flow).

7.2 Cơ chế hoạt động

```
// Content Script tạo <script> tag và chèn vào trang
function injectPageScript() {
  const script = document.createElement('script');
  script.textContent = `
    // Code này chạy TRONG JS world của Google Flow

    // Bước 1: Lưu lại fetch() gốc
    const originalFetch = window.fetch;

    // Bước 2: Thay thế fetch() bằng phiên bản "theo dõi"
    window.fetch = async function(...args) {
      // Gọi fetch() gốc bình thường
      const response = await originalFetch.apply(this, args);

      const url = typeof args[0] === 'string' ? args[0] : args[0]?.url;

      // Bước 3: Nếu request tới API Google → bắt response
      if (url?.includes('aisandbox-pa.googleapis.com')) {
        const clone = response.clone(); // Clone để không ảnh hưởng response gốc
        const data = await clone.json();

        // Bước 4: Nếu response chứa video URL → báo cho Content Script
        if (data.downloadUrl || data.videoUri) {
          window.dispatchEvent(new CustomEvent('__VEO_VIDEO_URL__', {
            detail: { url: data.downloadUrl || data.videoUri }
          }));
        }
      }

      return response; // Trả response bình thường cho Google Flow
    };
  `;
  document.documentElement.appendChild(script);
  script.remove(); // Xóa tag <script> (code đã execute rồi)
}
```

7.3 Giao tiếp MAIN World ↔ Content Script

```
┌──────────────────────┐          CustomEvent          ┌──────────────────────┐
│   MAIN World         │ ──────────────────────────→   │   Isolated World     │
│   (Injected Script)  │ '__VEO_VIDEO_URL__'           │   (Content Script)   │
│                      │                               │                      │
│   Override fetch()   │                               │   window.addEventListener(│
│   Clone response     │                               │     '__VEO_VIDEO_URL__', │
│   Parse JSON         │                               │     (e) => {         │
│   Emit CustomEvent   │                               │       capturedUrl = e.detail.url │
│                      │                               │     }                │
└──────────────────────┘                               └──────────────────────┘
```

Tại sao dùng CustomEvent? Vì đây là cách DUY NHẤT để 2 thế giới JS giao tiếp — chúng chia sẻ DOM nhưng không chia sẻ biến JS.

CHƯƠNG VIII: Side Panel UI — Bảng Điều Khiển

8.1 Layout từ screenshots thật

```
┌──────────────────────────────────────┐
│  🐹 VEO Automation                   │
│  Auto Veo on Google Flow   kylenguyen.me ↗ │
│  [👤 User Guide]  [🌐 English ▾]     │
├──────────────────────────────────────┤
│  ⚡ Control  |  ⚙️ Setting           │  ← 2 Tabs
├──────────────────────────────────────┤
│  Video Modes:                        │
│  [🎬 Text→Video] [🎞️ Frame→Video] [🧩 Ingredients→Video]│
│                                      │
│  Image Modes:                        │
│  [🖼️ Text→Image] [🔄 Image→Image] [🔗 Last Image→Image]│
├──────────────────────────────────────┤
│  💡 Tách nhiều prompts bằng dòng trống│
├──────────────────────────────────────┤
│  ⏱️ Duration per Prompt: [8s ▾]      │
├──────────────────────────────────────┤
│  📋 Queue:                           │
│  1. [🖼️] A cute 3D animated ...  371 chars [8s]│
│  2. [🖼️] A magical fantasy ...   319 chars [8s]│
│  3. [🖼️] A young ballerina ...  116 chars [8s]│
├──────────────────────────────────────┤
│  📊 Video Generation Progress:       │
│  ✅ Job 1 — A cute 3D ...  ▓▓▓▓▓▓▓▓ 100% │
│  ✅ Job 2 — A magical ...  ▓▓▓▓▓▓▓▓ 100% │
│  🔄 Job 3 — A young ...    ▓▒▒▒▒▒▒▒  26% │
└──────────────────────────────────────┘
```

CHƯƠNG IX: 10 Kỹ Thuật "Bí Kíp" — Tóm Tắt

#	Kỹ thuật	Vấn đề giải quyết	Mức độ
1	**React Native Setter**	Set value vào React input	🔴 Khó
2	**MutationObserver**	Chờ element render async	🟡 TB
3	**Dual-World Architecture**	Isolated vs MAIN world	🔴 Khó
4	**Selector Resilience**	Chống thay đổi class hash	🟡 TB
5	**Auto-Chaining**	Last frame → next prompt	🟡 TB
6	**SW Persistence**	Lưu state khi SW bị terminate	🔴 Khó
7	**Concurrency (Semaphore)**	Giới hạn 2 jobs song song	🟡 TB
8	**DataTransfer Upload**	Simulate file upload tự động	🔴 Khó
9	**Multi-Resolution DL**	Tải video nhiều chất lượng	🟢 Dễ
10	**i18n Dynamic Loading**	Đổi ngôn ngữ runtime	🟢 Dễ

Kỹ thuật #8: File Upload Automation (chưa nói ở trên)

```
// Simulate kéo-thả ảnh vào Google Flow
const dt = new DataTransfer();                          // Tạo "bàn tay ảo"
const file = await urlToFile(imageUrl, 'reference.png'); // Chuyển URL → File object
dt.items.add(file);                                     // "Cầm" file lên

const fileInput = document.querySelector('input[type="file"]');
fileInput.files = dt.files;                             // "Thả" file vào input
fileInput.dispatchEvent(new Event('change', { bubbles: true })); // Thông báo React
```

CHƯƠNG X: Xuất Bản Lên Chrome Web Store

📊 Diagram

```
graph LR
    subgraph "1. Phát triển"
        A["Viết code<br/>JS, HTML, CSS"] --> B["Build<br/>(npm run build)"]
        B --> C["Tạo file ZIP<br/>chứa manifest.json<br/>+ tất cả assets"]
```

```
    end

    subgraph "2. Nộp duyệt"
        C --> D["Upload ZIP lên<br/>Developer Dashboard"]
        D --> E["Điền thông tin:<br/>• Mô tả<br/>• Screenshots<br/>• Privacy policy"]
        E --> F["Submit"]
    end

    subgraph "3. Google Review (1-7 ngày)"
        F --> G{"Auto scan +<br/>Human review"}
        G -->|Pass| H["✅ Published!"]
        G -->|Fail| I["❌ Sửa lại"]
    end
```

- **Phí:** $5 một lần (đăng ký developer)
- **Review:** 1-7 ngày (trung bình 2-3 ngày)
- **Update:** Upload ZIP mới → tăng version trong manifest.json → submit lại

---

CHƯƠNG XI: Bài Tập & Câu Hỏi Ôn Tập 📝

Câu hỏi trắc nghiệm

1. Service Worker KHÔNG THỂ làm gì?

- A) Gửi message đến Content Script
- B) Dùng `document.querySelector()` ← ✅ **Đáp án**
- C) Lưu dữ liệu vào `chrome.storage`
- D) Tải file bằng `chrome.downloads`

2. Tại sao `textarea.value = "text"` không hoạt động với React?

- A) Textarea bị disabled
- B) React override setter và chỉ nhận synthetic events ← ✅ **Đáp án**
- C) Chrome chặn JavaScript
- D) Content Script không có quyền

3. MutationObserver dùng để làm gì?

- A) Thay đổi style của element
- B) Phát hiện khi DOM thay đổi ← ✅ **Đáp án**
- C) Gửi HTTP request
- D) Mã hóa dữ liệu

4. MAIN World injection cần thiết vì?

- A) Content Script không đọc được DOM

- B) Content Script không intercept được `window.fetch()` gốc ← ✅ **Đáp án**
- C) Chrome không cho phép content scripts
- D) Cần hiển thị popup

5. `chrome.storage.local` khác `localStorage` ở điểm nào?

- A) `chrome.storage` dùng được trong Service Worker (không có DOM) ← ✅ **Đáp án**
- B) Không có gì khác
- C) `localStorage` bảo mật hơn
- D) `chrome.storage` chỉ lưu được số

Bài tập thực hành

1. **Dễ:** Tạo file `manifest.json` cho extension hiển thị "Hello World" trong Side Panel
2. **Trung bình:** Viết `waitForElement()` function và test trên một trang web bất kỳ
3. **Khó:** Viết Content Script tự động điền text vào thanh tìm kiếm Google (dùng native setter trick)
4. **Nâng cao:** Tạo extension có Service Worker quản lý queue đơn giản (thêm/xóa items)

> 💡 **TIP**
>
> **Lộ trình học nếu muốn tự xây extension tương tự:**
>
> 1. 📖 Chrome Extension Getting Started
> 2. 🔍 Content Scripts + MutationObserver
> 3. ⚙️ Service Worker + Message Passing
> 4. 🖥️ Side Panel API
> 5. ⚛️ React DOM hacking (native setter trick)
> 6. 🔓 Fetch interception (MAIN world injection)

> **Hệ sinh thái 4 extensions cùng kiến trúc** của Truong Nguyen:

#	Extension	Platform
1	**VEO Automation**	Google Flow (VEO)
2	**Grok Automation**	grok.com

#	Extension	Platform
3	**Whisk Automation**	Google Whisk
4	**Meta AI Automation**	meta.ai

Cả 4 đều dùng chung **core architecture**: Side Panel + Service Worker + Content Script. Chỉ thay đổi **DOM selectors** cho từng platform → **template tái sử dụng**!